

Semi-Structured and Unstructured Data

CompTIA SecAI+: AI Security Certification Complete Exam Prep 2026 | Section 7: Data Types and Structures

1. The Data Structure Spectrum

Security data does not fall neatly into two categories. It occupies a spectrum from rigidly defined relational tables at one end to raw binary blobs at the other. Semi-structured data sits in the middle: it has some internal organisation — fields, tags, or key-value pairs — but does not conform to a fixed schema shared by all records. One JSON document in an alert stream may have seven fields; the next may have twelve, with three fields renamed and two entirely new ones added because the alert source was updated overnight.

This variability is not a bug in modern security data — it is a design reality. Security tools evolve, new data sources are added, and alert formats change with product versions. Architectures that require every record to conform to a rigid pre-declared schema break every time a vendor updates their log format. Semi-structured formats like JSON and XML were designed to accommodate this evolution while retaining enough structure for programmatic parsing.

2. JSON — The Dominant Security Data Exchange Format

JSON (JavaScript Object Notation) has become the default format for security data exchange between modern systems. It represents data as human-readable key-value pairs and supports nested structures: an alert object might contain a nested source object that itself contains IP address, geolocation, and ASN fields. JSON is self-describing — a parser reading a JSON document discovers its structure from the document itself, without consulting an external schema.

In practice, security practitioners encounter JSON in every layer of the stack: cloud security API responses, SIEM alert payloads, EDR telemetry, threat intelligence feeds, container runtime logs, and webhook notifications from security tools all use JSON. Proficiency in parsing JSON — using native language libraries in Python, JavaScript, or Go, or using JQ for command-line processing — is a foundational analyst skill. JSON path expressions allow extracting specific nested fields without parsing the entire document.

JSON Context	Example Fields	Security Relevance
Cloud security API response	resource_id, region, finding_type, severity, remediation	Ingest findings from AWS Security Hub or Azure Defender
EDR telemetry event	process_name, pid, parent_pid, cmdline, hash, user	Detect process injection and suspicious child processes
Threat intelligence feed	ioc_type, value, confidence, tlp, tags, expiry	Enrich alerts with IOC context from MISP or STIX
SIEM alert payload	rule_id, alert_name, entities, score, timestamp, status	Route and prioritise alerts in SOAR playbooks

3. XML — Legacy Standard with Enduring Presence

XML (Extensible Markup Language) predates JSON and uses a tag-based syntax similar to HTML. Where JSON uses braces and brackets, XML wraps content in opening and closing tags that can carry attributes: `<alert severity="high" source="firewall">...</alert>`. XML is more verbose than JSON for the same data but supports features that JSON lacks natively: namespaces for avoiding element name collisions across schemas, XPath for precise element extraction, and XSLT for transforming XML into other formats.

In security, XML remains prevalent in several domains that predate JSON's dominance. SAML (Security Assertion Markup Language) uses XML for authentication tokens exchanged between identity providers and service providers — understanding XML is therefore essential for anyone debugging SSO issues. STIX version 1.x used XML to exchange structured threat intelligence before migrating to JSON in version 2.0. Many enterprise security products — SIEMs, firewalls, and endpoint management platforms — export configuration and policy files in XML. Security professionals who dismiss XML as legacy technology miss its continued operational relevance.

4. Security Log Files — The Semi-Structured Middle Ground

Log files occupy an interesting position in the data structure spectrum. Traditional syslog messages have a partially consistent structure — priority, timestamp, hostname, and process name follow a standard format — but the message body that follows is free text whose structure depends entirely on the application that generated it. A Linux authentication event, a web server request, and a database query all write through the same syslog facility but produce incompatible message bodies.

Modern applications increasingly log in structured or semi-structured formats. JSON logging from application frameworks means each log line is a parseable JSON object with consistent field names across events of the same type. Web server logs (Apache Combined Log Format, nginx) follow consistent field patterns separated by spaces or quoted strings — structured enough to parse with a simple regex, but not a relational table. The practical challenge for security teams is building parsers that handle this diversity: extract the relevant fields from each source into a common internal format, gracefully handle malformed records, and deal with format changes when vendors update their products.

5. Parsing Techniques and Defensive Design

Parsing semi-structured data requires both technical capability and defensive design. A parser that crashes on malformed input becomes an availability risk in a high-volume security monitoring environment. A parser that silently drops records it cannot parse creates invisible gaps in monitoring coverage.

- JSON parsing — use native standard-library parsers (json module in Python, encoding/json in Go). Always handle parsing exceptions explicitly. Validate required fields are present before accessing them; missing fields are normal in semi-structured data.
- XML parsing — use established libraries (lxml in Python, javax.xml in Java). Be aware of XML External Entity (XXE) injection: a malicious XML document can reference external entities that cause the parser to fetch URLs or read local files. Disable external entity processing in all XML parsers used on untrusted input.

- Log parsing — use named capture groups in regular expressions rather than positional groups. Named groups remain correct even if the log format adds new fields before the captured field.
- Schema validation — even semi-structured data benefits from schema validation using JSON Schema or XML Schema Definition (XSD). Validate records against a schema to detect format changes early.
- Graceful degradation — parsers should log malformed records to a dead-letter queue for investigation rather than crashing or silently dropping them.

Learner warning — XXE injection: XML parsers that process untrusted input with external entity resolution enabled can be exploited to read files from the server's filesystem or make outbound network requests. This is a distinct vulnerability class from SQL injection but equally dangerous. Disable DOCTYPE declarations and external entity resolution in any parser handling untrusted XML.

6. Unstructured Data — Rich Information Without Schema

Unstructured data has no predefined internal organisation. It includes natural language text, images, audio recordings, video files, and binary executables. There are no column names, no data types, no foreign keys — just raw content. The challenge for security analysis is that unstructured data often contains the richest and most contextually nuanced information available: a security analyst's incident narrative captures judgment, context, and reasoning that no structured field can represent.

Unstructured security data includes: incident reports and analyst notes in case management systems, threat intelligence articles and security advisories, email content analysed for phishing indicators, chat communications collected during insider threat investigations, malware samples and binary files, network packet captures requiring deep inspection, and recorded security interviews or deposition transcripts. The volume of unstructured security data is enormous and growing as AI-assisted threat actors produce larger volumes of synthetic text and manipulated media.

7. Extracting Value from Unstructured Text

Natural Language Processing (NLP) is the primary tool for converting unstructured text into structured, queryable information. Several NLP techniques are directly applicable to security contexts.

NLP Technique	What It Does	Security Application
Named Entity Recognition (NER)	Identifies entities such as IP addresses, CVEs, organisation names, and hashes	Extract IOCs from threat reports without manual reading
Sentiment analysis	Classifies text as positive, negative, or neutral	Assess urgency in analyst notes; detect distress in insider threat communications
Topic modelling (LDA)	Discovers latent topics across a document corpus	Categorise unread security advisories by threat category
Summarisation	Condenses long documents into key points	Produce executive summaries of lengthy incident reports
Text classification	Assigns documents to predefined categories	Classify emails as phishing, legitimate, or suspicious

The output of NLP pipelines is structured data derived from unstructured inputs. Named entities extracted from a threat report — IP addresses, CVE identifiers, malware family names — can be inserted into a relational database and queried with SQL. This transformation from unstructured richness to structured queryability is precisely why organisations invest in NLP pipelines for security data: they convert unreadable volumes of text into actionable, searchable intelligence.

8. Binary Data and Malware Analysis

Binary data — executable files, compiled libraries, network packet captures, memory dumps, and encrypted archives — is entirely unstructured from a text perspective. Bytes carry meaning only when interpreted through the correct format specification. Analysing binary data requires specialised tools: disassemblers and decompilers for executables, packet analysers for network captures, hex editors for raw inspection, and format-aware parsers for file types like PE (Windows executables), ELF (Linux executables), and PCAP.

Machine learning applied to raw binary data has produced significant advances in malware detection. Models trained on byte sequences — treating executable content as a sequence of tokens similar to words in a document — can detect malware families without traditional signature databases, generalising to new variants that share underlying code structures with known malware. Deep learning approaches using convolutional neural networks applied to binary images (where each byte is mapped to a pixel) have demonstrated effectiveness at detecting malware across heterogeneous file formats.

Real-World Incident — SolarWinds SUNBURST (2020): Attackers inserted malicious code into a legitimate SolarWinds software update, creating a backdoored binary distributed to approximately 18,000 organisations. Traditional signature-based detection failed to identify the threat because the malware was signed with a legitimate SolarWinds certificate and disguised as a benign software component. The incident demonstrated that binary analysis beyond signature matching — including behaviour analysis and anomaly detection on binary characteristics — is essential for detecting sophisticated supply chain attacks embedded in legitimate software.

9. Unified Analysis — Handling Mixed Data Types

Real security investigations rarely involve a single data type. A phishing investigation might start with structured authentication logs showing a successful login from an unusual location, expand to semi-structured JSON EDR events showing the download of a suspicious file, then involve unstructured analysis of the phishing email body and binary analysis of the downloaded payload. Effective security architecture must handle this diversity without forcing analysts to switch between incompatible tools for each data type.

Data lakes address this by storing all data types in raw form, applying structure at query time rather than at ingestion. Unlike data warehouses that require a predefined schema, data lakes accept JSON, binary files, log files, and structured tables alike. Query engines like Apache Spark, Presto, and AWS Athena can query diverse data lake contents using SQL-like syntax, applying schema at read time. For security, this means storing everything — all log formats, all file types, all threat intelligence formats — without

transformation, and applying the appropriate parsing logic when building a specific detection or investigation query.

10. Search Technologies for Semi-Structured and Unstructured Data

Full-text search engines like Elasticsearch (the engine underlying many SIEMs and threat hunting platforms) invert the relationship between queries and data. Rather than scanning all records and filtering, they pre-build inverted indexes: for each word or token in the dataset, the index records which documents contain it. Queries then look up the index rather than scanning all data, producing millisecond responses over billions of documents.

Elasticsearch handles JSON natively, indexing all fields in a document automatically. It supports full-text search on unstructured fields (free-text matching) and exact-match queries on structured fields (IP address, hostname, severity) in the same query. Fuzzy matching finds documents with similar terms even when the exact spelling differs — useful for matching IOC strings that analysts may have transcribed with minor errors. Platforms like Splunk, OpenSearch, and Apache Solr provide similar capabilities with different trade-offs in scale, cost, and features.

11. Analogy — The Library and the Rubbish Bin

Think of structured data as a university library catalogue: every book is classified by subject, author, title, and ISBN. You can search precisely and retrieve exactly what you need. Semi-structured data is like the same library's collection of donated journals: each journal has a title and date on the cover, but the internal article structure varies by publication. Unstructured data is the donations pile in the corner: boxes of papers, handwritten notes, and recordings whose content is unknown until someone reviews each item individually.

The analogy extends to analysis tools: the library catalogue is queried with SQL, the donated journals are parsed with format-aware tools, and the donations pile requires NLP, OCR, and manual review to unlock its value. Security organisations with mature data strategies invest in all three — maintaining clean structured stores for high-speed detection, building parsers for semi-structured sources, and applying AI to extract intelligence from the unstructured pile that no human could read fast enough.

Key Terms Glossary

Term	Definition
Semi-structured data	Data with some organisation (fields, tags, key-value pairs) but no rigid schema shared by all records; examples include JSON and XML.
JSON	JavaScript Object Notation — a human-readable semi-structured format using key-value pairs and nested objects, dominant in modern security data exchange.
XML	Extensible Markup Language — a tag-based semi-structured format used in SAML, legacy STIX, and enterprise security product configurations.
Unstructured data	Data with no predefined internal organisation; includes natural

	language text, images, audio, video, and binary executables.
NER (Named Entity Recognition)	NLP technique that identifies and classifies entities such as IP addresses, CVEs, organisation names, and hashes within unstructured text.
XXE injection	XML External Entity injection — an attack exploiting XML parsers with external entity resolution enabled to read local files or make outbound network requests.
Data lake	A storage architecture accepting data of all types and formats in raw form, applying structure at query time rather than ingestion.
Full-text search	Search technology using inverted indexes to find documents containing specific words or phrases across large unstructured or semi-structured datasets.
STIX	Structured Threat Information eXpression — a standardised language for representing cyber threat intelligence; version 1.x used XML, version 2.0 uses JSON.
SAML	Security Assertion Markup Language — an XML-based standard for exchanging authentication and authorisation data between identity providers and service providers.
Dead-letter queue	A storage destination for messages or records that could not be processed successfully, enabling investigation and reprocessing without data loss.
Log parser	Software that extracts structured fields from semi-structured or unstructured log text using regex, grammar rules, or ML-based extraction.

Quick Revision Checklist

- Semi-structured data has internal organisation but no fixed schema shared by all records — JSON and XML are the primary examples in security contexts.
- JSON is the dominant modern security data exchange format; proficiency in JSON path expressions and parsing libraries is a foundational analyst skill.
- XML remains relevant in SAML authentication, legacy STIX threat intelligence, and enterprise security product configurations.
- Security log files vary from highly structured (JSON logging) to semi-structured (syslog) — parsers must handle format diversity gracefully and never silently drop malformed records.
- XXE injection exploits XML parsers with external entity resolution enabled — disable DOCTYPE declarations and external entities in parsers handling untrusted input.
- Unstructured data includes incident narratives, threat reports, email bodies, malware binaries, and packet captures — rich in context but not directly queryable.
- NLP techniques — NER, sentiment analysis, topic modelling, summarisation — convert unstructured text into structured, queryable security intelligence.
- Binary malware analysis uses disassemblers, deep learning on byte sequences, and behaviour analysis to detect threats that signature-based tools miss.

- Data lakes store all data types in raw form and apply schema at query time — the architectural choice for organisations handling diverse security data at scale.
- Full-text search engines like Elasticsearch power SIEMs by pre-building inverted indexes, enabling millisecond queries across billions of diverse log records.

10 Exam-Based MCQs — Semi-Structured and Unstructured Data

Test yourself after reading. These questions are written in real exam style — scenario-heavy, with plausible distractors. Read every explanation, including for questions you answered correctly.

Q1. Scenario: A CISO has prioritised long-term maintainability of the security monitoring architecture. The team expects the number of integrated tools to grow from 12 to 30 over the next three years. A security operations team is building a pipeline to ingest alerts from 12 different security tools. Each tool produces JSON alerts, but the field names and structures differ between tools and change with each product update. Which architectural approach BEST handles this long-term?

- A) Require all vendors to produce alerts conforming to a single fixed JSON schema before ingestion
- B) Store raw JSON in a data lake and apply format-specific parsing logic at query time for each tool
- C) Convert all JSON alerts to a relational database schema at ingestion time and reject records with unknown fields
- D) Store alerts as plain text files named by tool and date for manual analyst review

Answer: B **Explanation:** B is correct because a data lake stores raw data in its native format, applying parsing logic at query time. This accommodates format evolution without re-engineering the ingestion pipeline every time a vendor updates their alert structure. A is wrong because requiring vendors to conform to a fixed schema is impractical and breaks every time a vendor updates their product. C is wrong because rejecting records with unknown fields creates monitoring gaps; forcing all formats into a single relational schema at ingestion time is brittle to format changes. D is wrong because plain text files with manual review cannot scale to the alert volumes produced by 12 security tools.

Q2. Scenario: A red team exercise is assessing internal tooling built by the SOC team. One tool accepts XML file uploads from analysts for configuration processing. A developer is building an internal security tool that parses XML configuration exports from a firewall management platform. A penetration tester demonstrates that submitting a crafted XML file causes the parser to read the contents of /etc/passwd from the server running the tool. Which vulnerability class does this represent?

- A) SQL injection, because the XML data is passed to a database query
- B) XML External Entity (XXE) injection, because the parser resolves external entity references in the submitted XML
- C) Cross-site scripting (XSS), because the XML content is rendered in a web browser
- D) Path traversal, because the attacker specified a filesystem path in the request

Answer: B **Explanation:** B is correct because XXE injection exploits XML parsers with external entity resolution enabled. A malicious XML document can define an entity that references a local file path; when the parser resolves the entity, it reads and returns the file contents. A is wrong because SQL injection involves database queries, not XML parsing. C is wrong because XSS involves rendering unsanitised content

in a browser, not parsing XML files server-side. D is wrong because while the effect resembles path traversal, the mechanism is XML entity resolution specifically — classifying it as XXE is technically precise and the expected exam answer.

Q3. Scenario: *The analyst currently spends 15 hours per week manually extracting indicators from advisories. The CISO has asked for automation to reduce this to under 2 hours per week. A threat intelligence analyst receives 500 PDF security advisories per week and needs to extract IP addresses, domain names, and CVE identifiers for automatic enrichment of the SIEM. Which technique MOST efficiently accomplishes this at scale?*

- A) Train analysts to manually copy indicators from each advisory into a spreadsheet
- B) Use Named Entity Recognition (NER) to automatically identify and extract IP addresses, domains, and CVE identifiers from the text
- C) Convert all PDFs to XML format and use XPath queries to extract data
- D) Store the PDFs in a relational database and use full-text SQL LIKE queries to find indicator patterns

Answer: B **Explanation:** B is correct because NER is specifically designed to identify and classify entities such as IP addresses, CVE identifiers, and organisation names within unstructured text at scale. NER models trained on security text can process thousands of documents per hour. A is wrong because manual copying does not scale to 500 advisories per week and introduces transcription errors. C is wrong because converting PDFs to XML does not add semantic structure — the text remains unstructured inside the XML tags, and XPath queries cannot identify entity types. D is wrong because LIKE queries are extremely slow on large text fields, not a scalable approach, and cannot distinguish contextually valid IOCs from similar-looking non-IOC strings.

Q4. Which of the following BEST characterises semi-structured data?

- A) Data stored in tables with predefined columns and data types enforced by a relational database engine
- B) Data with some internal organisation such as key-value pairs or tags, but without a fixed schema shared by all records
- C) Binary data such as executable files and network packet captures with no human-readable structure
- D) Data that requires a schema defined in XSD before it can be stored or queried

Answer: B **Explanation:** B is correct because semi-structured data — exemplified by JSON and XML — has internal organisation that aids parsing, but records from the same source may have different fields, nested structures, or tag arrangements. A is wrong because predefined columns enforced by a database engine describes structured data in a relational database. C is wrong because binary data with no human-readable structure describes unstructured binary data, not the semi-structured middle ground. D is wrong because requiring an XSD schema before storage would make the data structured (schema-on-write), not semi-structured; semi-structured data typically applies schema at read time.

Q5. Why is full-text search technology like Elasticsearch particularly valuable for security monitoring?

- A) It enforces a fixed schema at ingestion, preventing malformed records from entering the dataset
- B) It pre-builds inverted indexes enabling millisecond queries across billions of diverse log records regardless of their structure
- C) It converts all semi-structured logs into normalised relational tables for SQL querying
- D) It provides encryption for log data at rest, satisfying compliance requirements automatically

Answer: B **Explanation:** B is correct because Elasticsearch's inverted index pre-computes which documents contain each token, enabling extremely fast lookups even across billions of records of varied format. A is wrong because Elasticsearch does not enforce a fixed schema; it accepts JSON documents with varying field sets and indexes them dynamically. C is wrong because Elasticsearch does not normalise logs into relational tables — it indexes JSON documents and supports field-level queries alongside full-text search. D is wrong because Elasticsearch does not provide encryption at rest by default; that capability exists through separate plugins or infrastructure-level encryption, and it is not the primary value proposition for security monitoring.

Q6. A security team uses a log parser that silently drops any log line that does not match its expected format. A vendor updates their firewall firmware, changing the log format slightly. What is the MOST significant security risk this creates?

- A) The parser will crash, causing a denial of service for the logging infrastructure
- B) The changed log format will consume more storage than the old format
- C) Security events logged in the new format will be silently dropped, creating invisible monitoring gaps
- D) The parser will trigger XXE injection against the firewall management interface

Answer: C **Explanation:** C is correct because a parser that silently drops unrecognised records creates monitoring blind spots — events that should generate alerts simply disappear without notification. This is particularly dangerous when format changes occur during security incidents. A is wrong because the described behaviour is silent dropping, not crashing; crashing would actually be more visible. B is wrong because log format changes typically have minimal storage impact and this is not a security risk. D is wrong because XXE injection affects XML parsers receiving untrusted XML, not log parsers processing plain text or JSON log lines from a firewall.

Q7. Which of the following is MOST accurate regarding SAML and its relationship to data formats?

- A) SAML uses JSON Web Tokens (JWTs) for authentication assertions between identity providers and service providers
- B) SAML uses XML to encode authentication assertions exchanged between identity providers and service providers
- C) SAML is a binary protocol that encodes authentication data in protobuf format for efficiency
- D) SAML was replaced by OpenID Connect in all modern enterprise environments

Answer: B **Explanation:** B is correct because SAML (Security Assertion Markup Language) uses XML to structure authentication assertions. Understanding XML is therefore essential for security professionals

debugging SSO integrations and SAML-based attacks such as XML signature wrapping. A is wrong because JWT is used in OAuth 2.0 and OpenID Connect, not SAML. C is wrong because SAML is an XML-based text protocol, not a binary protocol using protobuf. D is wrong because SAML remains widely deployed in enterprise SSO environments alongside OpenID Connect — it has not been replaced universally.

Q8. What distinguishes a data lake from a data warehouse in the context of security data management?

- A) Data lakes enforce a strict schema at ingestion time; data warehouses apply schema at query time
- B) Data lakes accept all data types in raw form and apply schema at query time; data warehouses require predefined schemas for all stored data
- C) Data lakes are only suitable for structured relational data; data warehouses support semi-structured and unstructured formats
- D) Data lakes use full-text search indexing; data warehouses use columnar storage for analytics

Answer: B **Explanation:** B is correct because a data lake stores data in its raw native format — JSON, binary, log files, relational tables — without requiring a predefined schema, applying structure at query time (schema-on-read). Data warehouses require structured data conforming to predefined schemas loaded through ETL pipelines. A is wrong because this reverses the actual definitions: schema-on-write applies to data warehouses and schema-on-read applies to data lakes. C is wrong because data lakes specifically excel at handling diverse formats including semi-structured and unstructured data, which is their main advantage over data warehouses for security use cases. D is wrong because both technologies can use columnar storage and full-text indexing; these are implementation details, not defining characteristics.

Q9. A malware analyst is examining a newly discovered executable that evades all signature-based antivirus tools. Which approach leverages machine learning on binary data to detect this threat?

- A) Extract strings from the binary and query them against a SQL database of known malware strings
- B) Convert the binary to an image where each byte maps to a pixel and apply a convolutional neural network trained on malware vs benign binary images
- C) Parse the binary as XML and use XPath to identify malicious function calls
- D) Decompress the binary using gzip and search for JSON payloads containing command-and-control URLs

Answer: B **Explanation:** B is correct because converting binaries to byte-level images and applying CNNs is an established machine learning approach to malware detection that captures structural patterns in binary code without relying on signatures. Models trained this way generalise to new malware variants that share code structure with known families, including polymorphic variants that evade signature matching. A is wrong because string matching against a database is signature-based, not ML-based, and would miss novel malware not in the database. C is wrong because executables are not XML documents and cannot be meaningfully parsed with XML tools. D is wrong because not all malware uses gzip compression or JSON-formatted C2 URLs; this describes a very specific and uncommon scenario.

Q10. A security analyst investigating a phishing campaign needs to correlate structured authentication logs, semi-structured JSON EDR events, and unstructured phishing email bodies. Which platform architecture BEST supports this multi-format investigation?

- A) A relational database that converts all data to normalised tables with a fixed schema before storage
- B) A data lake combined with a query engine that supports schema-on-read across structured, semi-structured, and unstructured data types
- C) Three separate specialised systems — one for each data type — with manual data transfer between systems
- D) A full-text search engine configured to index only the email bodies, with separate SQL queries for authentication and EDR data

Answer: B **Explanation:** B is correct because a data lake with a schema-on-read query engine (such as Apache Spark or AWS Athena) allows analysts to query authentication tables, JSON EDR events, and email text in a unified environment, joining across formats in a single investigation workflow. A is wrong because forcing all data into a fixed relational schema is brittle to format changes and loses nuance in unstructured data like email bodies. C is wrong because three separate systems with manual transfer introduces latency, version inconsistency, and operational complexity that hinder time-sensitive phishing investigations. D is wrong because splitting the data across three separate query interfaces (search engine for email, SQL for authentication, separate system for EDR) requires mental context switching and prevents unified correlation queries.